TEXT ONLY

Check for updates

From Function Frustrations to Framework Flexibility

FIXING TOOL CALLS WITH INDIRECTION

ERIK MEIJER

The current approach to function calling in LLMs (large language models) is brittle and inconsistent. Models often fail to determine when to invoke external tools reliably, generate invalid function calls, or account for tool outputs, leading to suboptimal interactions between users and models.

This unpredictability creates challenges for building reliable AI systems. Models may act overconfidently, bypassing tools for tasks they cannot perform well, or exhibit tool aversion based on biases from alignment training. Moreover, the inability to parameterize and abstract tool usage limits the flexibility and reusability of function-based workflows.

"All problems in computer science can be solved by another level of indirection." So goes the popular computer science aphorism. Applying this principle to function calling transforms the interaction into a robust, neuro-symbolic reasoning framework by introducing symbolic variable names and controlled dereferencing.

This approach not only addresses tool usage inconsistencies, but also enables parameterization and abstraction of interaction sequences, evolving

the framework into a fully fledged programming language. Leveraging symbolic reasoning enhances the interpretability, transparency, and reusability of LLM interactions, laying the groundwork for more reliable and sophisticated AI systems.

THE BASICS OF LLMS AND FUNCTION CALLS

LLMs function as compressed databases of world knowledge, trained and fine-tuned on diverse datasets to generate responses across a broad spectrum of queries.

A significant advance in LLM capabilities has come with the introduction of function-calling features in both closed and open foundation models.[1] Function calling allows developers to expose external tools and APIs through structured JSON (JavaScript Object Notation) schemas, which unlocks dynamic and realtime data sources that cannot be encoded in the model's weights or passed in via context. This includes realtime data such as weather and stock predictions as well as data that results from computations such as SQL queries or arbitrary code execution.[2]

While function calling enhances the capabilities and utility of LLMs, it also amplifies their main limitations because additional responsibilities are delegated to the model. Beyond the inherent stochastic nature of LLM predictions, these challenges include:

- Training data and alignment process biases.
- Difficulties establishing reasonable default assumptions aligned with human common sense.
- Potential exposure of sensitive information via function-call parameters and results.

- Risk of generating misleading or deceptive responses.

Building reliable and trustworthy systems becomes fundamentally difficult when the underlying AI components lack strict behavioral guarantees. These concerns become particularly pressing in autonomous or "agentic" applications built on top of LLMs, where the model's decisions can have direct real-world consequences without necessarily having humans in the loop.

This article examines the fundamentals of LLM function calling, presents concrete examples of the identified limitations, and proposes a pragmatic solution by selectively hiding information from the model through the addition of a level of indirection between values and their names.

Building reliable and trustworthy systems becomes fundamentally difficult when the underlying AI components lack strict behavioral guarantees.

HOW FUNCTION CALLS WORK

The function-calling protocol extends the standard `user>/assistant>` message exchange between client and LLM by introducing a number of new message roles that allow the model to invoke external tools dynamically through predefined function schemas. Each LLM inference API exposes function calling slightly differently, but the core mechanism is consistent across all major vendors. Without losing generality, this article provides an abstracted view that can easily be mapped onto any concrete implementation.

Let's examine the function-calling protocol using a concrete example, a Kotlin function that evaluates arithmetic expressions:

```
/**
* Evaluates a given mathematical [expression]
represented as a string.
*/
fun eval(expression: String): String = …
```

To make this function available to the LLM, its definition must be provided using JSON Schema:

```
{
“name”: “eval”,
 “description”: “Evaluates a given mathematical
  expression represented as a string”,
 “strict”: true,
 “parameters”: {
   “type”: “object”,
   “properties”: {
     “expression”: {
       “type”: “string”,
       “description”: “A mathematical
       expression represented as a string.”
     }
   },
   “strict”: true,
   “required”: [“expression”],
   “additionalProperties”: false
 }
}
```

Note that JSON Schema cannot specify return types, as it primarily validates JSON document structure and data types.

The interaction flow then proceeds by registering the available functions (here denoted by the separate `tools>` message, which is sometimes passed together with the user message and sometimes passed as an explicit `systems >` message) and a `user>` query that might require certain external tools to answer:

```
tools> … JsonSchema for fun eval(expression:
String): String …
user> What is the value of 1+2?
```

The model responds by generating an `assistant>` message that contains an optional explanation together with a function call, here denoted by the separate `calls>` message, but often is part of the assistant message, and a correlation ID:

```
assistant> … optional explanation …
calls> "12345" ↦ eval({ "expression": "1+2" })
```

This correlation ID system (represented by `"12345"` in the example) ensures that calls and their corresponding return values are properly matched, which is especially important when multiple function calls are made within a single interaction.

The client executes the function and returns the result via a `tool>` message using the same correlation ID, like so:

```
tool> "12345" ↦ { "result": 3 }
```

Once the model has gone through a number of tool-

calling turns (in this case, only one is required), it provides the answer based on the data it gathered from the function calls:

```
assistant> The result of (1+2) equals 3.
```

ISSUES WITH OPENAI FUNCTION CALLS

The function-calling protocol's design allows for both simplicity in basic scenarios and flexibility for more complex interactions by supplying configuration parameters to direct the function-calling decisions. As is explored in subsequent sections, however, this flexibility also introduces specific challenges in ensuring reliable and predictable behavior.

Thinking Silently

One significant issue is the model's tendency toward silent reasoning during function execution. In theory, the model's function calls should include both explanations and the calls themselves:

```
user> … question …
assistant> … explanation …
calls> … list of calls …
tool> … results …
⋮
assistant> … explanation …
calls> … list of calls …
tool> … results …
assistant> … answer …
```

In practice, however, the model often operates silently, executing multiple rounds of function calls without articulating its reasoning:

```
user> … question …
calls> … list of calls …
tool> … results …
⋮
calls> … list of calls …
tool> … results
assistant> … answer …
```

This silent chain of function calls is problematic. Without explanations between calls, it becomes difficult for the human user to understand the model's decision-making process. Without visibility into the model's thought process, there are fewer opportunities to correct errors and misconceptions and to guide the model toward better solutions.

While prompting techniques can encourage more verbose explanations, the model may still skip explaining certain steps in its reasoning chain. Hacks such as adding an "explanation" parameter to function calls often leads to superficial or vacuous comments that fail to provide a fluid account of the model's thought process.

Inflexible

The function-calling mechanism offers a number of configuration options to control its behavior, such as specifying a particular function to call, ensuring at least one function is called, or allowing any number of functions,

including none at all, to be invoked. These options have limited practical applicability, however, and letting the model choose remains the most effective approach.

Making the wrong choice about which specific tool to call leads to nonsensical function calls. For example, when forced to use the `eval` function for non-numeric queries, the system produces garbage results because the given tool cannot answer the question:

Making the wrong choice about which specific tool to call leads to nonsensical function calls.

```
user> what is the capital of the USA?
calls> eval({“expression”:“2 + 3”})
tool> 5
```

When you insist that the model use at least one tool, the tool calling won't terminate and the model will keep asking you to call some tool ad infinitum:

```
user> what is the capital of the USA?
calls> eval({“expression”:“2+3”})
tool> 5
calls> eval({“expression”:“11/2”})
tool> 5.5
calls> eval({“expression”:“3*4”})
```

The only feasible strategy is to let the model determine its own function usage. This approach alone works in general because predicting tool requirements for arbitrary queries is inherently difficult:

- You cannot know in advance if a tool is needed.
- The specific tool that is required may vary based on context.

- The number of interaction turns needed to answer a question is unpredictable.

The model, with its broader context understanding, is itself best positioned to make these decisions dynamically. But note that this knife cuts both ways. You cannot control whether the model will call a tool when you expect it to, but you also cannot control whether the model does *not* call a tool when you don't want it to.

Tool aversion

When allowing models to determine their function usage autonomously, you encounter a phenomenon called *tool aversion*. This manifests in two distinct ways: computational overconfidence and alignment-based resistance.

Computational overconfidence

Consider a simple arithmetic task: summing numbers from 10 down to 0. When asking the model to solve this challenge, it exhibits various behaviors that demonstrate its reluctance to use available tools.

Instead of using the provided `eval` tool to do arithmetic, the model often chooses to do "mental arithmetic" and computes the result directly itself:

```
tool> sum of 10, 9, ...,0
assistant> The sum of the numbers from 0 to 10
(10+9+ ... +0) is 55.
```

You can try to force your hand by prompting the model not to do this,[4] but then sometimes even when the

model does decide to use the tool, it generates invalid expressions such as including **"..."** in the following summation (yet embedded in JSON the call confirms according to the tool's schema):

```
user> sum of 10, 9, ..., 0
assistant> eval({"expression":"10+9+8+...+1+0"})
```

Other times, the model just brute forces it, which is also undesirable, as the model is still doing the majority of the arithmetic (counting down) itself:

```
user> sum of 10, 9, ..., 0
assistant> eval({"expression":
"10+9+8+7+6+5+4+3+2+1+0"})
```

In rare cases, the model gets clever and produces a solution as smart as young Gauss:

```
user> sum of 10, 9, ..., 0
assistant> eval({"expression": "10*(10+1)/2"})
```

And, if you are fortunate, it might generate the exact expression you were hoping for:

```
user> sum of 10, 9, ..., 0
assistant> eval({"expression":"(10 downTo
0).sum()" })
```

Even for this trivial example, it is nontrivial to make sure that the model employs what a human would think of

as common sense—namely, by using the calculator tool whenever it needs to do arithmetic.

Alignment-based resistance

While the model's hesitation to use a tool for arithmetic might be attributed to overconfidence, the next example of tool aversion uncovers a deeper problem—namely, that the model's alignment fundamentally biases its decision to invoke a tool in certain situations. This more subtle form of tool aversion emerges when the model faces ethically charged queries. This behavior reveals how the model's alignment training influences its willingness to defer to external tools. To show this effect, let's introduce a more open-ended tool that purposefully stresses the model's alignment and inner convictions— an oracle designed to provide guidance on complex personal decisions:

```
tools> … JsonSchema for fun oracle(conundrum:
String): String …
```

For complex but ethically permissible decisions, the model tends to defer to the oracle as expected:

```
user> [Query about a complex personal medical
decision]
calls> oracle({"conundrum":"[Query details]"})
```

When faced with queries that conflict with its alignment training, however, the model actively resists using the oracle tool, instead providing direct guidance based on the hidden ethical and legal frameworks with which it is aligned:

```
user> [Query about a procedure with estab-
lished ethical/legal concerns]
assistant> [Direct response citing ethical/le-
gal frameworks without oracle consultation]
```

This selective tool aversion demonstrates how the model's underlying alignment training can override its general instruction to use available tools. This behavior, while potentially desirable in some contexts, raises questions about sovereignty over the behavior of agentic systems built on top of foundation models.

Ignoring function results

Besides an aversion to call tools, models also exhibit unpredictable behavior when handling function-call results. In some cases, even if the model generates a function call, it may disregard the tool's output entirely if it thinks the answer is incorrect:

```
user> 1+2
calls> eval({"expression": "1+2" })
tool> 0
assistant> The result of the expression (1+2)
is 3.
```

Here the tool returned 0 as the result of computing 1+2, but the model silently decided to ignore the function's result and substitute its own answer. Again, you can resort to prompting to mitigate this behavior, but of course there is no guarantee that these instructions will always be followed.

In particular, in more complex scenarios the model may selectively interpret, question, or qualify tool responses depending on whether the answer aligns with its values:

```
user> [Query regarding a complex and nuanced
societal concept]
assistant> To address the complexity and
diversity surrounding this topic, let's consult
the oracle for a more comprehensive under-
standing.
calls> oracle({"conundrum": "[Query regarding
societal concept]"})
tool> [Context-specific response]
assistant> The oracle provides the following
response: [Context-specific response]. However,
it's worth noting that this is a complex topic
with diverse perspectives, and interpretations
may vary depending on cultural, societal, or
individual contexts."
```

The model's inconsistent behavior when choosing tools and taking their results into account highlights the key challenge with tool use: While you must rely on the model to decide when and how to use tools, it may not always align with your expectations, despite your best efforts. Whether because of confidence in its own capabilities, misjudging the utility of a tool, or simply generating imprecise outputs, the model simply does not always use tools in an optimal way.

ADDING A LEVEL OF INDIRECTION

Once again invoking the famous aphorism, "All problems

While you must rely on the model to decide when and how to use tools, despite your best efforts it may not always align with your expectations.

in computer science can be solved with an extra level of indirection," we can apply this principle directly to the challenges of tool-calling identified earlier in this article. Rather than allowing the model to manipulate values directly, you can add a layer of symbolic reasoning by introducing names that represent values. The actual dereferencing of names by the model is reserved for highly controlled scenarios and can be implemented by providing the model with a dereference function. By not showing concrete values, you can force the model to reason purely symbolically; hence giving it less information to eschew function calls, or interpret their results. When AI models are used as function evaluators, they should behave like traditional computers, executing instructions without questioning how or why.

As a benefit of introducing names for concrete values, *parameterization* of message sequences occurs for free. Furthermore, allowing the naming of these parameterized prompts results in *abstraction*. With abstraction and parameterization in place, the framework evolves into a fully fledged programming language, unlocking powerful new possibilities for structured reasoning and interaction on top of the weak foundations of function calling.

To implement symbolic reasoning, a new parameter to each function is introduced: a `result` parameter that specifies a name for the result of that function invocation. Consider the running example `eval`:

```
{
 "name": "eval",
 "description": "Evaluates a given mathematical
```

```
  expression represented as a string",
  "strict": true,
  "parameters": {
    "type": "object",
    "properties": {
      "expression": {
        "type": "string",
        "description": "A mathematical
         expression represented as a string."
      },
      "result": {
        "type": "string",
        "description": "Descriptive unique
         variable @name for the result of
         evaluating the function."
      }
    },
    "strict": true,
    "required": [ "expression", "result" ],
    "additionalProperties": false
  }
}
```

Note that, in principle, this additional parameter isn't strictly necessary, as you could use the correlation ID to represent the name for the result value instead. However, since there is no control over how correlation IDs are generated, and because the resulting code needs to remain readable and intuitive for human users, introducing this explicit parameter is a worthwhile tradeoff. Providing meaningful and descriptive names also enhances the model's capability to reason symbolically.

To ensure variable names are easily distinguishable, we adopt a simple convention: Prefix them with @. This keeps variable usage visually distinct and consistent, enhancing readability.

When executing a sequence of tool calls, you can maintain an environment of name-to-value bindings on the user side. Assistant messages are rendered for the user by substituting the actual values of variables in the response using the environment (which the model does not see):

```
user> 1+2
calls> eval({“expression”:“1+2”,“result”:“@Sum”})
tool> @Sum
assistant> The sum of 1 + 2 is @Sum=3.
user> 2*@Sum
calls> eval({“expression”:“2*@Sum”,“result”:“@
DoubleSum”})
tool> @DoubleSum
assistant> The result of multiplying 2 by
`@Sum=3` is @DoubleSum=6.
```

Introducing this level of indirection transforms tool-calling from a brittle mechanism into a more robust, neuro-symbolic reasoning framework. In particular, as mentioned earlier, interaction sequences can now be parameterized and given names:

```
/**
* Compute the mean @Mean of @X and @Y.
*/
```

```
mean(@X: Number, @Y: Number, @Mean: Number) :-
   user> What is the mean of @X and @Y?
   assistant> To compute the mean, we first
    compute the sum of @X and @Y
   calls>eval({"expression":"@X+@Y","result":
    "@SumXY"})
   assistant> Then we divide the sum @SumXY
    by 2 to find the mean
   calls>eval({"expression":
    "@SumXY/2","result":"@Mean"})
   assistant>The mean of @X and @Y is
     @Mean.
```

These reusable definitions are a superset of MCP (Model Context Protocol) prompts.[3] Both accept parameters and chain multiple interactions with the LLM, but MCP prompts do not capture tool calls.

AND ANOTHER INDIRECTION

The latest hype in LLMs surrounds models such as OpenAI's o1 that are specifically trained for chain-of-thought reasoning. But the actual chain-of-thought reasoning is often hidden, and only the final answer is returned to the user.

These "test-time compute" models are advertised as more powerful than previous-generation "next token prediction" models, raising the obvious question of whether they should be leveraged for function-calling scenarios. Unfortunately, as of spring 2025, many reasoning models struggle to break down the final solution it presents into small steps connected by function calls.

Rather than relying on prompt engineering to direct the model's behavior, adding another layer of indirection proves to be a much simpler solution.[5] First, the reasoning model creates a plan based on the user's question, using information about available tools but without generating tool calls. Next, a regular model uses the plan as input to answer the question, inserting tool calls at the relevant points.

When switched to JSON instead of text output, the model does produce a sequence of tool calls, but explaining those calls becomes even harder than with regular models.

CONCLUDING THOUGHTS

Function-calling capabilities directly enhance the utility of language models, allowing them to access dynamic, realtime data sources and perform complex computations. As demonstrated here, however, the current state of the art in tool-calling[7] suffers from significant inconsistencies. Models exhibit overconfidence, avoid tools unnecessarily, or produce invalid or suboptimal interactions. These issues weaken the reliability and transparency that developers need when building robust compound AI systems.

The principle of indirection can be applied to introduce a paradigm shift: replacing direct value manipulation with symbolic reasoning using named variables. This simple yet powerful trick directly resolves inconsistencies in tool usage and enables parameterization and abstraction of interactions. The transformation of function calls into reusable and interpretable frameworks elevates tool calling into a neuro-symbolic reasoning framework. This approach unlocks new possibilities for structured interaction and dynamic AI systems.

This establishes a more reliable, transparent, and expressive interface that connects language models

with the external tools they use, grounded in sound programming language principles.[6]

References

1. Function calling. Hugging Face; https://huggingface.co/docs/hugs/en/guides/function-calling.
2. Patil, S. G., Zhang, T., Wang, X., Gonzalez, J. E. 2023. Gorilla: Large language model connected with massive APIs. arXiv:2305.15334v1; https://arxiv.org/abs/2305.15334.
3. Prompts. Model Context Protocol; https://modelcontextprotocol.io/docs/concepts/prompts.
4. Sreenivasan, K., Chen, J., Koppol, P., Turok, E., Foley-Cox, B., Qureshi, A., Havens, S. 2024. Beyond the leaderboard: unpacking function calling evaluation. Databricks; https://www.databricks.com/blog/unpacking-function-calling-eval.
5. Tam, Z. R., Wu, C.-K., Tsai, Y.-L., Lin, C.-Y., Lee, H.-y., Yun-Nung Chen, Y.-N. 2024. Let me speak freely? A study on the impact of format restrictions on performance of large language models. *Proceedings of the Conference on Empirical Methods in Natural Language Processing: Industry Track,* 1218–1236; https://aclanthology.org/2024.emnlp-industry.91.pdf.
6. Tennent, R. D. 1977. Language design methods based on semantic principles. *Acta Informatica* 8, 97–112; https://link.springer.com/article/10.1007/BF00289243.
7. Zhong, L., Du, Z., Zhang, X., Hu, H., Tang, J. 2025. ComplexFuncBench: exploring multi-step and constrained function calling under long-context scenario. arXiv:2501.10132v1; https://arxiv.org/abs/2501.10132v1.

Erik Meijer *brings a rare combination of technical expertise and people leadership to his latest quest to use AI to democratize end-user programming. As a renowned computer scientist, entrepreneur, and tech influencer, Meijer has made pioneering contributions to programming languages, compilers, cloud infrastructures, and AI throughout his tenures at Microsoft, Meta (Facebook), Utrecht University, and Delft University of Technology.*

Copyright © 2025 held by owner/author. Publication rights licensed to ACM.